Godot interfaces

Often one needs scripts that rely on other objects for features. There are 2 parts to this process:

1. Acquiring a reference to the object that presumably has the features.
2. Accessing the data or logic from the object.

The rest of this tutorial outlines the various ways of doing all this.

Acquiring object references

For all Objects, the most basic way of referencing them is to get a reference to an existing object from another acquired instance.

GDScript C#

```
var obj = node.object # Property access.
var obj = node.get_object() # Method access.
```

The same principle applies for RefCounted objects. While users often access Node and Resource this way, alternative measures are available.

Instead of property or method access, one can get Resources by load access.

GDScript C#

```
# If you need an "export const var" (which doesn't exist), use a conditional
# setter for a tool script that checks if it's executing in the editor.
# The `@tool` annotation must be placed at the top of the script.
@tool

# Load resource during scene load.
var preres = preload(path)
# Load resource when program reaches statement.
var res = load(path)

# Note that users load scenes and scripts, by convention, with PascalCase
# names (like typenames), often into constants.
const MyScene = preload("my_scene.tscn") # Static load
const MyScript = preload("my_script.gd")

# This type's value varies, i.e. it is a variable, so it uses snake_case.
@export var script_type: Script

# Must configure from the editor, defaults to null.
@export var const_script: Script:
    set(value):
        if Engine.is_editor_hint():
            const_script = value

# Warn users if the value hasn't been set.
func _get_configuration_warnings():
    if not const_script:
        return ["Must initialize property 'const_script'."]

    return []
```

Note the following:

1. There are many ways in which a language can load such resources.
2. When designing how objects will access data, don't forget that one can pass resources around as references as well.
3. Keep in mind that loading a resource fetches the cached resource instance maintained by the engine. To get a new object, one must duplicate an existing reference or instantiate one from scratch with `new()`.

Nodes likewise have an alternative access point: the SceneTree.

GDScript C#

```
extends Node

# Slow.
func dynamic_lookup_with_dynamic_nodepath():
    print(get_node("Child"))

# Faster. GDScript only.
func dynamic_lookup_with_cached_nodepath():
    print($Child)

# Fastest. Doesn't break if node moves later.
# Note that `@onready` annotation is GDScript-only.
# Other languages must do...
#     var child
#     func _ready():
#         child = get_node("Child")
@onready var child = $Child
func lookup_and_cache_for_future_access():
    print(child)

# Fastest. Doesn't break if node is moved in the Scene tree dock.
# Node must be selected in the inspector as it's an exported property.
@export var child: Node
func lookup_and_cache_for_future_access():
    print(child)

# Delegate reference assignment to an external source.
# Con: need to perform a validation check.
# Pro: node makes no requirements of its external structure.
#      'prop' can come from anywhere.
var prop
func call_me_after_prop_is_initialized_by_parent():
    # Validate prop in one of three ways.

    # Fail with no notification.
    if not prop:
        return

    # Fail with an error message.
    if not prop:
        printerr("'prop' wasn't initialized")
        return

    # Fail and terminate.
    # NOTE: Scripts run from a release export template don't run `assert`s.
    assert(prop, "'prop' wasn't initialized")

# Use an autoload.
# Dangerous for typical nodes, but useful for true singleton nodes
# that manage their own data and don't interfere with other objects.
func reference_a_global_autoloaded_variable():
    print(globals)
    print(globals.prop)
    print(globals.my_getter())
```

Accessing data or logic from an object

Godot's scripting API is duck-typed. This means that if a script executes an operation, Godot doesn't validate that it supports the operation by **type**. It instead checks that the object **implements** the individual method.

For example, the CanvasItem class has a `visible` property. All properties exposed to the scripting API are in fact a setter and getter pair bound to a name. If one tried to access CanvasItem.visible, then Godot would do the following checks, in order:

- If the object has a script attached, it will attempt to set the property through the script. This leaves open the opportunity for scripts to override a property defined on a base object by overriding the setter method for the property.
- If the script does not have the property, it performs a HashMap lookup in the ClassDB for the "visible" property against the CanvasItem class and all of its inherited types. If found, it will call the bound setter or getter. For more information about HashMaps, see the data preferences docs.
- If not found, it does an explicit check to see if the user wants to access the "script" or "meta" properties.
- If not, it checks for a `_set` / `_get` implementation (depending on type of access) in the CanvasItem and its inherited types. These methods can execute logic that gives the impression that the Object has a property. This is also the case with the `_get_property_list` method.
    - Note that this happens even for non-legal symbol names, such as names starting with a digit or containing a slash.

As a result, this duck-typed system can locate a property either in the script, the object's class, or any class that object inherits, but only for things which extend Object.

Godot provides a variety of options for performing runtime checks on these accesses:

- A duck-typed property access. These will be property checks (as described above). If the operation isn't supported by the object, execution will halt.

  GDScript C#

```
# All Objects have duck-typed get, set, and call wrapper methods.
get_parent().set("visible", false)

# Using a symbol accessor, rather than a string in the method call,
# will implicitly call the `set` method which, in turn, calls the
# setter method bound to the property through the property lookup
# sequence.
get_parent().visible = false

# Note that if one defines a _set and _get that describe a property's
# existence, but the property isn't recognized in any _get_property_list
# method, then the set() and get() methods will work, but the symbol
# access will claim it can't find the property.
```

- A method check. In the case of CanvasItem.visible, one can access the methods, `set_visible` and `is_visible` like any other method.

  GDScript C#

```
var child = get_child(0)

# Dynamic lookup.
child.call("set_visible", false)

# Symbol-based dynamic lookup.
# GDScript aliases this into a 'call' method behind the scenes.
child.set_visible(false)

# Dynamic lookup, checks for method existence first.
if child.has_method("set_visible"):
    child.set_visible(false)

# Cast check, followed by dynamic lookup.
# Useful when you make multiple "safe" calls knowing that the class
# implements them all. No need for repeated checks.
# Tricky if one executes a cast check for a user-defined type as it
# forces more dependencies.
if child is CanvasItem:
    child.set_visible(false)
    child.show_on_top = true

# If one does not wish to fail these checks without notifying users,
# one can use an assert instead. These will trigger runtime errors
# immediately if not true.
assert(child.has_method("set_visible"))
assert(child.is_in_group("offer"))
assert(child is CanvasItem)

# Can also use object labels to imply an interface, i.e. assume it
# implements certain methods.
# There are two types, both of which only exist for Nodes: Names and
# Groups.

# Assuming...
# A "Quest" object exists and 1) that it can "complete" or "fail" and
# that it will have text available before and after each state...

# 1. Use a name.
var quest = $Quest
print(quest.text)
quest.complete() # or quest.fail()
print(quest.text) # implied new text content

# 2. Use a group.
for a_child in get_children():
    if a_child.is_in_group("quest"):
        print(quest.text)
        quest.complete() # or quest.fail()
        print(quest.text) # implied new text content

# Note that these interfaces are project-specific conventions the team
# defines (which means documentation! But maybe worth it?).
# Any script that conforms to the documented "interface" of the name or
# group can fill in for it.
```

- Outsource the access to a Callable. These may be useful in cases where one needs the max level of freedom from dependencies. In this case, one relies on an external context to setup the method.

GDScript C#

```
# child.gd
extends Node
var fn = null

func my_method():
    if fn:
        fn.call()

# parent.gd
extends Node

@onready var child = $Child

func _ready():
    child.fn = print_me
    child.my_method()

func print_me():
    print(name)
```

These strategies contribute to Godot's flexible design. Between them, users have a breadth of tools to meet their specific needs.